Message #2 (from S:>kmp>mail.kbin.newest)
Date: Mon, 26 Aug 1991 16:28 EDT
From: David A. Moon <moon@cambridge.apple.com>
To: kmp@Symbolics.com
Subject: Ralph Conditions (part 2 of 2)

>>> Miscellaneous condition operators

DISPLAY of a condition prints the condition's "error message" with no additional text, punctuation, or newlines. Many condition classes define methods for DISPLAY. If you define your own condition class, you need to define a method for DISPLAY unless the inherited method is suitable. If you have a condition object and wish to print its error message, you can call DISPLAY.

[Using DISPLAY corresponds to Common Lisp's condition system, and provides a very convenient interface to ~A in FORMAT. However, it might not be what we want. This issue should be revisited once the output architecture of Ralph has been specified. For now, DISPLAY is here as a placeholder since there has to be some way to get the "error message."]

In addition, some condition classes define accessor functions. The accessor functions for the standardized classes are specified briefly with the class above. None of these are SET!-able.

>>> SPECIFICATION: ADDITIONAL CONVENIENCE FEATURES

>>> Full set of operators for signalling

The first operator listed here is a repeat of the basic operator. The others all have straightforward implementations in terms of other primitives, but are provided to enable concise code in simple, common cases. The first eight operators listed consist of four pairs, where the two members of each pair have the same name but different arguments; in other words a pair of operators is one function that can be called in two ways, either with a condition object or with an error message constructed from a FORMAT string and arguments.

All operators listed in this section are sealed functions.

SIGNAL condition
SIGNAL string {argument}*

Signal the condition, trying each active dynamic handler, most recent first. If all dynamic handlers decline, calls (DEFAULT-HANDLER condition). If a handler returns, all the values that it returned are returned from SIGNAL. If SIGNAL returns when the condition's recovery protocol does not allow returning, some handler has violated protocol; SIGNAL does not check for this error. If the condition is a restart, the caller of SIGNAL should always assume that it might return.

The second form signals a condition of type <SIMPLE-WARNING>. There is a default handler that displays the warning in a user-interface dependent way and then returns #f.

See note below about the "dynamically active handler" concept.

ERROR condition
ERROR string {argument}*

Similar to SIGNAL but never returns; if a handler returns, ERROR bombs directly to the debugger. ERROR is used to make it clear that a program does not expect to receive control again after signalling a condition, and might enable the compiler to generate slightly more compact code.

The second form signals a condition of type <SIMPLE-ERROR>. There is a default handler that will invoke the debugger (which might be an interactive debugger, a "core dumper", a "bomb icon", or something else) if no dynamic handler handles this condition.

CERROR restart-description condition
CERROR restart-description string {argument}*

Same as ERROR but first establishes a handler for <SIMPLE-RESTART> whose description is restart-description, a string or a function of one argument.

If the restart handler is invoked, CERROR returns #f. Otherwise CERROR never returns. If CERROR returns, the program should take the corrective actions promised in the restart-description. CERROR is the standard way to signal correctable errors when no special class of restart condition is required.

BREAK condition
BREAK [string {argument}*]

Obtains a condition in the same way as SIGNAL but then calls the debugger immediately without signalling first. BREAK establishes a <SIMPLE-RESTART> so the debugger can continue execution. This is useful for breakpoints. BREAK always returns #f. With no arguments, a default message string is used.

CHECK-TYPE value type

Signals a <TYPE-ERROR> if (INSTANCE? value type) is false. If Ralph defines types that are more general than just classes, CHECK-TYPE will work with them.

This is a function, unlike in Common Lisp where it is a macro that can SET! value to a replacement value if a <NEW-VALUE> restart is signalled.

>>> Additional operators for handling

HANDLER-CASE protected-form {((type #key condition test description) body...)}* [Macro]

Establishes handlers during the dynamic extent of the evaluation of the protected-form. The first clause is the outermost of these handlers. If one of these handlers is invoked, it never declines, but immediately takes a non-local exit, executes the body forms in the dynamic environment that was in effect when the HANDLER-CASE was entered, and returns the values of the last body form, or #f if the body is empty. If the protected-form completes execution, HANDLER-CASE returns the values of the protected-form. Note that when the body forms of a clause are executed, the handlers established by the HANDLER-CASE are no longer active.

Type, test, and description are as for HANDLER-BIND. Condition, if present, is not evaluated and is the name of a variable that is bound to the condition around the body forms. HANDLER-CASE can expand into HANDLER-BIND and BIND-EXIT, it exists only for program conciseness. It is likely that HANDLER-CASE will be more commonly used than HANDLER-BIND.

DEFAULT-HANDLER condition [Generic Function]

Called if no dynamic handler handles the condition. There are predefined methods for <CONDITION>, which returns #f, <SERIOUS-CONDITION>, which calls the debugger, <WARNING>, which prints the message and returns #f, and <RESTART>, which signals an error.

RESTART-QUERY restart [Generic Function]

This generic function is defined for all subclasses of <RESTART>. It engages the interactive user in a dialog and stores the results in slots of restart. This is to be called from a handler, after making a restart and before signalling it. The debugger uses this, for example. There is a default method for <RESTART> which does nothing, for restart classes that do not need any interaction before restarting.

RETURN-QUERY condition [Generic Function]

If the recovery protocol of this condition allows returning values, this engages the interactive user in a dialog and returns the results as any number of values, which the handler should return. RETURN-QUERY should not be called if RETURN-ALLOWED? returns #f.
If you define your own condition class whose recovery protocol allows returning values, you need to define a method for RETURN-QUERY unless the inherited method is suitable.

>>> Operators for introspection

DO-HANDLERS funarg [Sealed Function]

Apply funarg to all dynamically active handlers, most recently established first. Funarg receives four arguments: type, test, function, and description.

```
The arguments describe a dynamically active handler.
All arguments have dynamic extent and must not be modified.
Test is defaulted to a function that always returns #t, but description
is #f if unsupplied by the handler, or else a function or a string.

RETURN-ALLOWED? condition                                    [Generic Function]

This predicate returns #t if the recovery protocol of this condition allows
returning values, or #f if it does not.  There is a default method for <CONDITION>
that returns #f.  If you define your own condition class, you need to define a
method for RETURN-ALLOWED? unless the inherited method is suitable.

RETURN-DESCRIPTION condition                                 [Generic Function]

If the recovery protocol of this condition allows returning values, this
returns a description of the meaning of returning values, similar to the
description of a restart handler.  This can be a function, a string, or #f.
RETURN-DESCRIPTION should not be called if RETURN-ALLOWED? returns #f.
If you define your own condition class whose recovery protocol allows
returning values, you need to define a method for RETURN-DESCRIPTION unless
the inherited method is suitable.


>>> EXAMPLES

;;; Classes such as <file-not-found> used in these examples are
;;; invented for the example and are not part of the specification

;;; The DISPLAY methods herein could change to something else when
;;; that issue is resolved

;;; This example shows minimal handling of a file-not-found error

(handler-case (open "file-that-doesnt-exist")
  ((<file-not-found> condition: c)
   (format *error-output* "~&The file ~A was not found."
           (file-name c))))

;;; This example shows how to handle a file-not-found error by
;;; reading a different file instead.

(handler-bind (<file-not-found>
               (method (condition next-handler)
                 (signal (make <try-a-different-file>
                               file-name: "my-emergency-backup-file"))))
  ....
  (open "file-that-doesnt-exist")
  ....)

(define-method open (the-file)
  (handler-case (guts-of-open the-file)
    ((<try-a-different-file>
      description: (method (stream)
                     (format stream "Read a different file instead of ~A" the-file))
      condition: restart)
     (open (file-name restart)))))

(define-method guts-of-open (the-file)
  (let ((result (operating-system-open the-file)))
    (cond ((instance? result <stream>) result)
          ((id? result +file-not-found-error-code+)
           (error (make <file-not-found> file-name: the-file)))
          ...)))

(define-class <file-not-found> (<error>)
  ((file-name init-keyword: file-name:)))

(define-method display ((self <file-not-found>) stream)
  (format stream "The file ~A was not found" (file-name self)))

(define-class <try-a-different-file> (<restart>)
  ((file-name init-keyword: file-name:)))

;;; This is the same example improved so the restart handler that reads another
;;; file can only be reached by a handler for the associated condition, useful
;;; if there are nested errors.
```

```
(handler-bind (<file-not-found>
               (method (condition next-handler)
                 (signal (make <try-a-different-file>
                               condition: condition
                               file-name: "my-emergency-backup-file"))))
  ....
  (open "file-that-doesnt-exist")
  ....)

(define-method open (the-file)
  ....
  (guts-of-open the-file))

(define-method guts-of-open (the-file)
  (bind ((result (operating-system-open the-file)))
    (cond ((instance? result <stream>) result)
          ((id? result +file-not-found-error-code+)
           (bind ((condition (make <file-not-found> file-name: the-file)))
             (handler-case (error condition)
               ((<try-a-different-file>
                 test: (compose (curry id? condition) restart-condition)
                       #+ignore ;; For mortals, the above means:
                       (method (cond)
                         (id? condition (restart-condition cond)))
                 description: (method (stream)
                                (format stream "Read a different file instead of ~A"
                                        the-file))
                 condition: restart)
                (open (file-name restart))))))
          ...)))

(define-class <file-not-found> (<error>)
  ((file-name init-keyword: file-name:)))

(define-method display ((self <file-not-found>) stream)
  (format stream "The file ~A was not found" (file-name self)))

(define-class <try-a-different-file> (<restart>)
  ((condition init-keyword: condition: reader: restart-condition)
   (file-name init-keyword: file-name:)))
```

>>> NOTE

Note about the "dynamically active handler" concept:

In the absence of CALL-WITH-CURRENT-CONTINUATION, the "dynamically active handler" concept can be understood in terms of the stack model of control, as in such languages as Common Lisp and C++. However, this does not mean that the Ralph condition system depends on the stack model, nor that the condition system would break if CALL-WITH-CURRENT-CONTINUATION were added to Ralph.

Conditions can be understood in terms of the continuation model. Each function call can be regarded as having two continuations, the normal continuation of Scheme and the exception continuation, which comprises the dynamically active condition handlers. A function call passes the caller's exception continuation to the callee. HANDLER-BIND and HANDLER-CASE are the only constructs that create new exception continuations. CALL-WITH-CURRENT-CONTINUATION creates a function that encapsulates both continuations; calling this function calls the saved normal continuation with the saved exception continuation as its exception continuation. With these semantics, everything works as intuitively expected.